

Módulo 12 – Capítulo 01 – Sección 01

Propósito del proyecto final: demostrar dominio integrado de AI Engineering aplicado

El Módulo 11 cerró con una pregunta que todo ingeniero de IA enfrenta cuando su organización alcanza un nivel de madurez suficiente: ¿cómo se gobiernan, escalan y mantienen sistemas de IA en contextos enterprise reales, con restricciones de cumplimiento regulatorio, heterogeneidad de fuentes y deuda técnica acumulada? El Módulo 12 responde esa pregunta de la única forma que produce aprendizaje durable: construyendo. El proyecto final no es un ejercicio de laboratorio desconectado del contexto organizacional; es el caso de uso que cualquier equipo de ingeniería podría encontrar en una empresa de tecnología mediana: un asistente técnico inteligente para ingenieros de software, capaz de consultar la base de conocimiento interna de la organización. Los aspectos enterprise que están presentes en este proyecto incluyen control de acceso por roles, trazabilidad de auditoría, restricciones de costo operativo y criterios de seguridad no negociables. Los aspectos que se simplifican explícitamente por razones pedagógicas son la integración con sistemas legacy (SAP, ERPs) y el cumplimiento GDPR completo, que requieren infraestructura organizacional fuera del alcance de un proyecto formativo.

El proyecto final integra cada capa técnica estudiada a lo largo de los once módulos anteriores. No es una suma de partes: es el lugar donde las decisiones de diseño interactúan entre sí y producen consecuencias que solo son visibles cuando el sistema opera como un todo. Elegir un modelo de embedding con menor dimensionalidad tiene efecto en la calidad del retrieval, que tiene efecto en la faithfulness de las respuestas, que tiene efecto en la tasa de bypasses detectados en el red teaming. Diseñar el agente con herramientas de solo lectura es simultáneamente una decisión de funcionalidad y una decisión de seguridad. Ningún capítulo anterior podía enseñar esas interacciones en profundidad — el proyecto final es el espacio donde emergen y donde el ingeniero aprende a gestionarlas.

A diferencia de los ejercicios parciales de cada capítulo, el proyecto final exige que el ingeniero tome decisiones arquitectónicas reales con restricciones reales: latencia máxima aceptable, costo por petición, controles de seguridad no negociables y métricas de calidad medibles. El

sistema deberá incluir un pipeline RAG con evaluación RAGAS, un agente con herramientas instrumentadas, observabilidad con OpenTelemetry y despliegue automatizado mediante CI/CD. El propósito es demostrar que el ingeniero puede construir, evaluar y operar un sistema de IA en producción, no solo prototiparlo.

Antes de comenzar el diseño, vale la pena anclar los criterios de éxito que este módulo verificará al final. El Capítulo 10 contiene la rúbrica de evaluación completa, pero sus criterios clave informan cada decisión desde el inicio: el pipeline RAG debe alcanzar RAGAS faithfulness ≥ 0.82 , el agente debe completar tareas con task completion rate $\geq 75\%$ en el golden dataset, el sistema debe superar un red teaming de seguridad con tasa de bypass $< 5\%$, y la latencia P95 del pipeline completo debe ser inferior a 3 segundos. Conocer estos criterios antes de la primera línea de código no limita el diseño — lo orienta.

Capacidades demostradas al completar el proyecto

- **Integración de sistema cohesivo:** RAG + agente + API con contratos de interfaz definidos entre componentes que evolucionan de forma independiente.
- **Evaluación cuantitativa:** RAGAS (faithfulness, answer relevance, context precision, context recall) sobre golden dataset anotado manualmente, con pipeline de evaluación continua integrado en el CI/CD.
- **Instrumentación completa:** OpenTelemetry con trazas distribuidas por etapa del pipeline, métricas de latencia P50/P95/P99 y logs estructurados correlacionados en Grafana.
- **Controles de seguridad activos:** validación de inputs multicapa, output filtering con detección de PII y protección contra prompt injection directa e indirecta.
- **Despliegue reproducible:** Docker multi-stage, pipeline CI/CD con gate de evaluación y despliegue en Kubernetes con gestión de secrets via AWS Secrets Manager.

Nota del Arquitecto: La tentación al comienzo de un proyecto integrador es empezar por el componente más familiar — generalmente el LLM y el prompt. Resiste esa tentación. Los proyectos de IA que fracasan en producción casi nunca fallan por la calidad del prompt; fallan por ausencia de evaluación, por secrets en el código, por ausencia de runbooks cuando algo falla a las 2am. Empieza por los criterios de éxito y por la estructura del repositorio — son los que determinan si el sistema sobrevive al primer mes en producción.

El proyecto final es la demostración de que se puede diseñar, construir, evaluar y operar un sistema de IA completo — no solo hacer funcionar un LLM en un notebook. Los capítulos que siguen construyen ese sistema capa por capa, comenzando por las decisiones arquitectónicas que documentamos en los ADRs del Capítulo 2.

Para recordar: El proyecto final integra los once módulos del libro en un sistema con restricciones reales. Los criterios de éxito — faithfulness ≥ 0.82 , task completion $\geq 75\%$, bypass $< 5\%$, latencia P95 $< 3s$ — definen qué significa "terminado" antes de la primera línea de código.

Módulo 12 – Capítulo 01 – Sección 02

El problema a resolver: definición del caso de uso, alcance y restricciones técnicas

¿Cuánto tiempo pierde un ingeniero de software buscando en Confluence la configuración correcta de un servicio, el ADR que documenta por qué se eligió una base de datos, o el runbook del incidente que ocurrió hace tres meses con síntomas similares? En organizaciones de tamaño mediano con varios años de documentación acumulada, la respuesta es entre 30 minutos y 2 horas por semana por ingeniero. El asistente técnico que construye este proyecto final ataca ese problema directamente: un sistema capaz de responder preguntas sobre la base de conocimiento interna de la organización usando documentación técnica, ADRs, runbooks y tickets históricos como fuentes primarias, con respuestas que citan la fuente y pueden trazarse al documento original.

El caso de uso tiene una definición precisa porque la precisión es la que hace posible comprometerse con métricas. El sistema responde preguntas sobre documentación técnica interna — no genera código arbitrario, no ejecuta comandos en sistemas externos y no accede a bases de datos de producción. Esta delimitación no es una limitación de ambición; es una decisión de seguridad y de alcance que hace posible operar el agente con nivel de autonomía controlado. Un agente que puede ejecutar comandos en sistemas externos requiere un modelo de amenazas completamente diferente, controles mucho más rigurosos y un proceso de aprobación human-in-the-loop para cada acción de escritura. Al limitar el agente a consultas de lectura sobre una base de conocimiento interna, el sistema puede operar de forma autónoma con un nivel de riesgo aceptable para el equipo.

Las fuentes de datos incluyen documentación técnica en Markdown (archivos `.md` en repositorios Git), especificaciones de API en OpenAPI YAML, runbooks en Confluence exportados como HTML, y ADRs en formato Markdown. El alcance incluye también tickets históricos de incidentes con resolución documentada, que son especialmente valiosos para diagnóstico de problemas recurrentes. Las fuentes excluidas explícitamente son: código fuente de repositorios (riesgo de filtración de lógica de negocio sensible), emails y canales de

Slack (contenido no estructurado sin proceso de validación editorial), y documentos marcados como confidenciales por el equipo de seguridad.

Las restricciones técnicas del sistema son cuantitativas y verificables, no aspiracionales. La latencia de respuesta inferior a 3 segundos para el percentil P95 es la restricción más exigente: incluye el tiempo de embedding de la query, búsqueda en Qdrant, reranking con Cohere, razonamiento del agente y generación de respuesta con GPT-4o. El costo máximo de 0.02 USD por petición es una restricción económica que impone disciplina en la elección del modelo y en el tamaño del contexto inyectado al LLM. La capacidad de manejar hasta 50 consultas concurrentes sin degradación visible impone requisitos de configuración de Uvicorn y de gestión de conexiones a los servicios externos. Estas tres restricciones interactúan entre sí: reducir la latencia suele requerir más paralelismo, que incrementa el costo; aumentar la calidad del retrieval suele requerir más chunks en el contexto, que incrementa la latencia y el costo.

Restricciones técnicas definidas

- **Latencia P95 < 3 segundos:** para el pipeline completo — retrieval + reranking + generación LLM — medido bajo carga de 50 usuarios concurrentes con ramp-up de 10 segundos.
- **Costo por petición < 0.02 USD:** contabilizando tokens de prompt y completion con GPT-4o, costo de embedding con text-embedding-3-small, costo de reranking con Cohere y costo de infraestructura prorrateado.
- **Concurrencia mínima de 50 peticiones simultáneas:** con throughput sostenido sin memory leaks detectables en un test de 30 minutos de duración.
- **Superficie de ataque limitada:** el agente no ejecuta código arbitrario ni accede a sistemas externos en escritura; todas las herramientas son de solo lectura sobre la base de conocimiento interna.
- **SLA de disponibilidad del 99.5%:** con health checks activos cada 30 segundos y restart automático mediante Kubernetes liveness probes ante fallos de proceso.

Nota del Arquitecto: Las restricciones que se definen en esta sección son las que el equipo tendrá que defender ante stakeholders cuando el sistema esté en producción. Si no puedes medir una restricción con un test automatizado, no puedes garantizarla en un SLA. "Rápido" no es una restricción — "latencia P95 < 3 segundos bajo carga

de 50 usuarios" sí lo es. El nivel de especificidad importa porque determina si el sistema puede ser aprobado para producción con evidencia o solo con confianza.

Definir explícitamente qué queda fuera del alcance técnico es tan importante como definir qué está dentro. Las exclusiones del sistema — generación de código arbitrario, ejecución de comandos en sistemas externos, acceso a documentos confidenciales — son las que hacen posible comprometerse con las métricas de seguridad del red teaming. Sin exclusiones claras, el alcance crece durante el desarrollo y los criterios de seguridad pierden precisión. El Capítulo 2 documenta estas decisiones de alcance en el ADR-003, que registra el nivel de autonomía máximo del agente como una decisión arquitectónica explícita con justificación de seguridad.

Para recordar: Definir explícitamente qué queda fuera del alcance técnico es tan importante como definir qué está dentro — las restricciones son las que hacen posible comprometerse con métricas medibles y verificables.

Módulo 12 – Capítulo 01 – Sección 03

Arquitectura de alto nivel: componentes, flujos de datos y estructura del repositorio

Un sistema de IA en producción puede describirse en términos de sus componentes, pero solo se comprende en términos de sus flujos de datos. La pregunta que la arquitectura de alto nivel debe responder no es "¿qué piezas tiene el sistema?" sino "¿cómo viaja la información desde el documento original hasta la respuesta del usuario?" Esa pregunta tiene dos respuestas correspondientes a los dos flujos principales del sistema: el flujo de ingesta, que transforma documentos en vectores indexados, y el flujo de consulta, que transforma la pregunta de un usuario en una respuesta fundamentada en el conocimiento disponible.

El flujo de ingesta es asíncrono y tolerante a fallos. Un usuario o un proceso automatizado envía un documento al endpoint `/ingest` de la API; el endpoint valida el payload, encola la tarea en Redis mediante Celery y devuelve un `task_id` de seguimiento. El worker Celery procesa la tarea en background: el documento pasa por el parser correspondiente a su tipo (LlamaParse para PDF, python-markdown para Markdown, BeautifulSoup para HTML de Confluence), luego por el chunker RecursiveCharacterTextSplitter configurado a 512 tokens con 64 de overlap, luego por el embedder text-embedding-3-small de OpenAI, y finalmente el vector resultante se hace upsert en la colección `knowledge_base` de Qdrant junto con sus metadatos de payload. Un hash SHA-256 del contenido del documento detecta documentos duplicados o sin cambios, evitando re-indexaciones costosas.

El flujo de consulta es síncrono y evaluable. El usuario envía una petición al endpoint `/query` con su query en texto libre; el middleware de autenticación verifica el JWT y extrae los claims de autorización; el InputValidator aplica tres capas de validación (schema Pydantic, lista negra de injection, clasificador de intent); el agente LangGraph recibe la query validada y comienza su ciclo ReAct: llama a `search_knowledge_base`, que internamente embedda la query con text-embedding-3-small, ejecuta búsqueda híbrida RRF en Qdrant y aplica reranking con Cohere Rerank sobre los top-20 resultados; el agente recibe los top-5 chunks, razona sobre ellos y decide si necesita búsquedas adicionales o puede construir la respuesta;

GPT-4o genera la respuesta final citando las fuentes; el OutputFilter escanea la respuesta en busca de PII antes de devolverla al usuario.

La arquitectura sigue un patrón de capas bien definidas con interfaces explícitas entre cada capa. Esto no es solo una decisión estética: permite reemplazar cualquier componente sin afectar al resto. Si se necesita cambiar text-embedding-3-small por un modelo de embedding fine-tuned, solo cambia el implementador de la interfaz de embedding — el agente, el chunker y el filtro de output no saben nada del cambio. Si se necesita sustituir Cohere por un reranker local, solo cambia el reranker — el pipeline de búsqueda híbrida lo llama con la misma interfaz.

Antes de escribir la primera línea de código, el repositorio debe tener una estructura de directorios que refleje la arquitectura del sistema. Esta estructura actúa como mapa del código para cualquier ingeniero que llegue al proyecto:


```

technical-assistant/
├── docs/
│   ├── adr/                # ADR-001 al ADR-004
│   └── architecture.md     # Diagrama Mermaid de alto nivel
├── src/
│   ├── api/                # FastAPI: endpoints, middleware, schemas
│   ├── agent/              # LangGraph: grafo, nodos, herramientas
│   ├── rag/                # Pipeline RAG: ingesta, chunking, retrieval
│   ├── security/           # InputValidator, OutputFilter, auth
│   └── observability/      # OpenTelemetry instrumentation
├── tests/
│   ├── unit/               # Tests por componente con mocks
│   ├── integration/        # Tests con dependencias reales (Qdrant test)
│   └── evaluation/         # Scripts RAGAS, golden dataset, red teaming
├── infra/
│   ├── terraform/          # Módulos networking, compute, services
│   └── k8s/                # Manifests Kubernetes y Argo Rollouts
├── .github/
│   └── workflows/          # CI/CD pipeline con stages build/test/evaluate/deploy
├── docker-compose.yml      # Stack de desarrollo local completo
├── Dockerfile              # Multi-stage build para producción
└── .env.example            # Variables de entorno documentadas

```

Esta estructura separa el código de aplicación (`src/`), los tests por tipo, la infraestructura como código (`infra/`) y la documentación de decisiones (`docs/adr/`). Cada directorio tiene una responsabilidad única y sus dependencias fluyen en una sola dirección: `api/` depende de `agent/` y `security/`, `agent/` depende de `rag/`, `rag/` no depende de ningún módulo interno — solo de servicios externos.

Componentes principales del sistema

- **Ingesta asíncrona:** parser de documentos por tipo MIME, chunker `RecursiveCharacterTextSplitter`, embedder `text-embedding-3-small`, upsert en Qdrant con idempotencia por hash SHA-256 del contenido. Cola Celery + Redis para procesamiento tolerante a fallos.

- **Base vectorial Qdrant:** colección `knowledge_base` con índices HNSW ($m=16$, $ef_construction=100$), payload indexes en `document_type`, `team` e `ingested_at`, y vectores sparse BM25 para soporte de búsqueda híbrida nativa.
- **Pipeline de recuperación:** búsqueda híbrida RRF ($k=60$) sobre embeddings densos y BM25 sparse, reranking Cohere rerank-english-v3.0 sobre top-20, selección de top-5 y compresión de contexto con LLMChainExtractor.
- **Agente ReAct con LangGraph:** grafo de estado con nodos `reason`, `act`, `observe` y `respond`, herramientas `search_knowledge_base`, `get_document_by_id` y `list_available_sources`, `max_iterations=5` con fallback a respuesta parcial.
- **API REST FastAPI:** endpoints `/query`, `/ingest` y `/health`, middleware de autenticación JWT RS256, rate limiting por usuario, InputValidator multicapa y OutputFilter con detección de PII.

Nota del Arquitecto: La estructura de directorios del repositorio comunica las decisiones arquitectónicas antes de que el lector abra un solo archivo. Un directorio `security/` al mismo nivel que `agent/` dice "la seguridad es un componente de primera clase, no un afterthought". Un directorio `evaluation/` con los scripts RAGAS dice "la evaluación es parte del proceso de desarrollo, no una verificación final". Dedicar 15 minutos a diseñar esta estructura antes del primer commit ahorra semanas de refactoring posterior.

La arquitectura de alto nivel debe resolverse en papel antes de escribir la primera línea de código. Cada componente introduce latencia, costo y complejidad operativa que deben justificarse explícitamente. Las decisiones técnicas más impactantes — modelo de embedding, framework agéntico, base vectorial — se documentan en los ADRs del Capítulo 2, que registran no solo qué se eligió sino por qué, qué alternativas se evaluaron y qué consecuencias se anticipan.

Para recordar: La arquitectura de alto nivel y la estructura del repositorio son artefactos de diseño previos a la implementación — el árbol de directorios es el mapa del sistema que permite a cualquier ingeniero orientarse desde el primer día.

Módulo 12 – Capítulo 01 – Sección 04

Stack tecnológico: selección justificada de modelos, bases de datos, frameworks y herramientas

Elegir un stack tecnológico para un sistema de IA en producción es una de las decisiones con mayor costo de reversibilidad del proyecto. Cambiar el modelo de embedding después de indexar 50.000 documentos requiere re-indexar toda la colección. Cambiar el framework agéntico después de implementar el grafo de estado implica reescribir la lógica de razonamiento. Por eso la selección del stack no puede basarse en popularidad, familiaridad o "lo que todos usan" — debe basarse en criterios técnicos medibles que correspondan a las restricciones del sistema: latencia P95, costo por petición, operabilidad en Kubernetes y soporte de las capacidades necesarias para cada componente.

El modelo de lenguaje principal es GPT-4o (gpt-4o-2024-08-06), elegido mediante un benchmark interno de 100 preguntas del dominio técnico objetivo. Los tres candidatos evaluados fueron GPT-4o, Claude 3.5 Sonnet (Anthropic) y Gemini 1.5 Pro (Google). Los criterios de eliminación temprana descartaron a Gemini 1.5 Pro por inconsistencias en el soporte de function calling estructurado en el momento de la evaluación. El benchmark entre GPT-4o y Claude 3.5 Sonnet midió accuracy semántica evaluada por LLM-as-judge (0.88 vs 0.86 respectivamente), latencia TTFT (time to first token) y costo por 1000 tokens de output. GPT-4o fue elegido por combinar el mejor score de accuracy con soporte nativo para parallel tool calling, que el agente ReAct necesita para ejecutar múltiples herramientas en un solo turno del ciclo de razonamiento. La decisión se documenta en el ADR-001 del Capítulo 2 con los datos completos del benchmark.

Para embeddings, text-embedding-3-small (1536 dimensiones, 0.00002 USD por 1000 tokens) fue seleccionado sobre text-embedding-3-large (3072 dimensiones, 0.00013 USD por 1000 tokens) y sobre el modelo text-embedding-ada-002 que era la opción por defecto anterior. La comparación midió recall@5 en un conjunto de 500 pares query-documento del dominio: text-embedding-3-small alcanzó 0.82, text-embedding-3-large alcanzó 0.86, y ada-002 alcanzó 0.76. La diferencia de 0.04 puntos en recall@5 entre small y large no justifica un costo 6.5 veces mayor por token de embedding, especialmente considerando que el reranking

posterior recupera parte del recall perdido en la recuperación inicial. La elección de ada-002 fue descartada por su inferior rendimiento en el benchmark del dominio.

Qdrant v1.9 fue seleccionado como base vectorial sobre ChromaDB y Pinecone. ChromaDB fue descartado por la ausencia de soporte de búsqueda híbrida nativa y sus limitaciones de escalabilidad en colecciones grandes. Pinecone fue evaluado y descartado por tres razones: su modelo serverless no permite despliegue local para desarrollo (lo que complica el setup local sin credenciales de producción), sus filtros por payload tienen un costo adicional por operación en el plan básico, y su API no soporta la configuración granular de índices HNSW que el sistema requiere. Qdrant soporta búsqueda híbrida nativa (sparse + dense) con fusión RRF sin configuración adicional, permite despliegue Docker para desarrollo y Qdrant Cloud para producción con la misma API, y ofrece cuantización escalar que reduce el uso de memoria un 75% con penalización de recall inferior al 2%.

LangGraph fue elegido sobre LangChain AgentExecutor y AutoGen para implementar el agente. LangChain AgentExecutor no soporta grafos de estado con ciclos condicionales, lo que limita la implementación del patrón ReAct a flujos lineales sin retroalimentación. AutoGen fue evaluado pero descartado por la complejidad de configuración de sus conversaciones multi-agente para un caso de uso de agente único y por la menor integración con el ecosistema LangChain que el resto del pipeline RAG usa. LangGraph es el único que soporta grafos de estado con ciclos, checkpointing nativo para reanudación de conversaciones y control granular del flujo de ejecución — exactamente lo que el ciclo ReAct requiere.

Selección justificada del stack por componente

- **LLM principal:** GPT-4o (gpt-4o-2024-08-06) — context window de 128k tokens, JSON mode nativo, parallel tool calling; accuracy 0.88 en benchmark de dominio; latencia P95 < 2s medida bajo carga de 10 peticiones simultáneas.
- **Embeddings:** text-embedding-3-small (OpenAI) — recall@5 de 0.82 en benchmark de dominio, costo 6.5x menor que text-embedding-3-large, 62% de mejora sobre ada-002 en MTEB; 1536 dimensiones compatibles con cuantización escalar de Qdrant.
- **Base vectorial:** Qdrant v1.9 — búsqueda híbrida nativa (sparse + dense), filtros por payload con índices explícitos, cuantización escalar, despliegue Docker para dev y Qdrant Cloud para producción con la misma API REST.
- **Framework agéntico:** LangGraph — grafos de estado con ciclos condicionales, checkpointing en PostgreSQL para persistencia de sesión, control granular de condiciones de parada y fallback ante max_iterations.

- **Framework de API:** FastAPI + Uvicorn — generación automática de OpenAPI 3.1 desde schemas Pydantic, middleware de autenticación JWT, prometheus_fastapi_instrumentator para métricas, 4 workers en producción.
- **Infraestructura:** Docker Compose para desarrollo local, Kubernetes (EKS) para producción, Terraform para IaC, Argo Rollouts para canary deployment, Grafana + Tempo + Prometheus + Loki para observabilidad.

***Nota del Arquitecto:** Documentar la alternativa evaluada y descartada para cada componente del stack es tan importante como documentar la elección final. Sin ese registro, el equipo repetirá la evaluación en cada revisión de arquitectura porque no habrá memoria de por qué se descartó ChromaDB o por qué se eligió LangGraph sobre AutoGen. Los ADRs del Capítulo 2 son el lugar donde este razonamiento se preserva formalmente — este capítulo de diseño es la base de datos mental sobre la que se construyen.*

La selección del stack tecnológico es una decisión que debe tomarse una vez con evidencia y revisarse periódicamente con los mismos criterios. Los modelos fundacionales evolucionan: GPT-4o-2024-08-06 puede ser superado por una versión posterior en accuracy o en costo, y esa revisión debe hacerse con el mismo benchmark del dominio que produjo la elección original. El ADR-001 del Capítulo 2 establece el proceso de revisión: cuándo revisar (cuando el proveedor lanza una versión nueva o cuando las métricas de calidad en producción caen por debajo del umbral), cómo revisarlo (mismo benchmark de 100 preguntas del dominio) y quién toma la decisión final.

Para recordar: Documentar la alternativa evaluada y descartada para cada componente del stack es parte de la decisión técnica — sin ese registro, el equipo repetirá la evaluación en cada revisión de arquitectura.

Módulo 12 – Capítulo 01 – Sección 05

Criterios de éxito técnico: métricas de calidad, rendimiento y seguridad del sistema

Un sistema de IA cuyo éxito se mide por "funcionar bien" no puede evaluarse, no puede mejorarse y no puede aprobarse para producción con rigor. La pregunta que define la madurez de un equipo de ingeniería de IA no es "¿el sistema responde?" sino "¿qué evidencia tenemos de que el sistema responde correctamente, dentro del tiempo acordado, al costo comprometido y sin comportamientos de seguridad inaceptables?" Los criterios de éxito del proyecto final son la respuesta operativa a esa pregunta: métricas verificables con scripts de evaluación, no aspiraciones interpretadas subjetivamente por cada miembro del equipo.

La dimensión de calidad RAG se mide con el framework RAGAS sobre un golden dataset de 200 pares pregunta-respuesta-contexto anotados manualmente. El sistema debe alcanzar RAGAS faithfulness ≥ 0.85 sobre el golden dataset completo y ≥ 0.82 como umbral de gate en el CI/CD para evaluar cada deploy. Esta diferencia entre el umbral de producción (0.85) y el gate del CI/CD (0.82) es intencional y no una inconsistencia: el gate del CI/CD opera sobre 20 muestras aleatorias del golden dataset (no sobre las 200) y con mayor tolerancia estadística para evitar falsos bloqueos por variabilidad de muestreo. El umbral de producción es más exigente porque se calcula sobre el dataset completo con mayor estabilidad estadística. El Capítulo 6 explica esta lógica al introducir el gate de CI/CD. Además del faithfulness, se mide answer relevance (umbral ≥ 0.80), context precision (≥ 0.75) y context recall (≥ 0.70).

La dimensión de rendimiento establece contratos de latencia verificables con Locust. La latencia P50 debe ser inferior a 1.5 segundos, P95 inferior a 3 segundos y P99 inferior a 5 segundos, todos medidos bajo una carga de 50 usuarios concurrentes con ramp-up de 10 segundos. Estos umbrales se eligieron basándose en investigación de UX sobre tolerancia humana a la espera en interfaces de asistencia técnica: los usuarios toleran hasta 3 segundos de latencia para consultas de documentación cuando perciben que el sistema está "pensando" (lo que un spinner comunica efectivamente), pero abandonan o reformulan la consulta si la espera supera los 5 segundos. El costo por petición debe mantenerse por debajo de 0.015 USD

en promedio, con un máximo de 0.03 USD para queries de alta complejidad con contexto extenso que requieran múltiples iteraciones del agente.

La dimensión de seguridad tiene criterios binarios y cuantitativos. El red teaming de 50 ataques documentados debe producir una tasa de bypass inferior al 5% — es decir, máximo 2 o 3 ataques exitosos de los 50 ejecutados. Los ataques que logran bypass se convierten en casos de test de regresión: el sistema debe fallar esos tests antes de corregir los controles, y luego pasar esos tests después de la corrección. Todos los inputs deben ser validados mediante Pydantic con rechazo de payloads malformados antes de alcanzar el modelo — esta validación es verificable con tests unitarios del InputValidator. La autorización por roles debe garantizar que un usuario sin acceso a ADRs confidenciales no pueda recuperarlos mediante consultas creativas — verificable con tests de integración que usan tokens JWT con `allowed_document_types` restringidos.

La dimensión de operabilidad incluye criterios de continuidad de servicio: tiempo de recovery ante fallo de pod inferior a 30 segundos (medido desde el crash hasta que el replacement pod responde al health check), zero downtime en despliegue canary con rollback automático disponible, y un checklist de producción de 30 puntos completado y firmado antes del primer despliegue. La existencia del checklist es en sí misma un criterio de éxito — es la demostración de que el equipo ha pensado sistemáticamente en cada dimensión antes de poner el sistema en manos de los usuarios.

Métricas de éxito por dimensión

- **Calidad RAG:** RAGAS faithfulness ≥ 0.85 (producción) / ≥ 0.82 (gate CI/CD), answer relevance ≥ 0.80 , context precision ≥ 0.75 , context recall ≥ 0.70 ; todos sobre golden dataset de 200 casos anotados.
- **Rendimiento:** latencia P50 $< 1.5s$, P95 $< 3s$, P99 $< 5s$ bajo carga de 50 usuarios concurrentes; throughput mínimo de 20 req/s en steady state; degradación de P95 inferior al 20% al pasar de 25 a 50 usuarios.
- **Costo:** media por petición < 0.015 USD con breakout de costos de embedding (OpenAI), reranking (Cohere) y generación (GPT-4o); máximo < 0.03 USD para queries de alta complejidad.
- **Seguridad:** tasa de bypass de prompt injection $< 5\%$ en red teaming de 50 ataques documentados; 0 secrets en el código verificado con detect-secrets en el pre-commit hook; 0 CVEs críticos en imagen Docker verificado con Trivy.

- **Operabilidad:** recovery time < 30s ante fallo de pod; zero downtime en canary deployment; checklist de producción de 30 puntos completado; runbook revisado por al menos un ingeniero externo al equipo de desarrollo.

***Nota del Arquitecto:** Hay una tensión real entre los criterios de faithfulness del golden dataset (0.85) y los del gate CI/CD (0.82). En mi experiencia, los equipos que usan el mismo umbral en ambos lugares acaban paralizando el pipeline CI/CD por variabilidad estadística — evaluar 20 muestras produce mayor varianza que evaluar 200. La solución correcta no es bajar el umbral de producción sino aceptar que el gate de CI/CD es una señal de alerta temprana con mayor tolerancia, no la evaluación definitiva. El deploy que pasa el gate con 0.83 todavía necesita validación completa con el golden dataset de 200 pares antes de promover a producción.*

Los criterios de éxito deben definirse antes de comenzar la implementación — una métrica que no puede medirse durante el desarrollo no puede garantizarse en producción. Pero tienen un segundo propósito igual de importante: son el lenguaje compartido del equipo. Cuando el equipo de producto pregunta "¿está listo para producción?", la respuesta no es "creo que sí" — es "el checklist de producción tiene 28 de 30 puntos completados, faithfulness es 0.83 con tendencia positiva, y el gate de red teaming pasó con tasa de bypass del 4%". Ese nivel de precisión en la comunicación del estado del sistema es en sí mismo una capacidad de ingeniería. El Capítulo 10 introduce la rúbrica de evaluación formal que formaliza estos criterios como instrumento de evaluación del proyecto.

Para recordar: Los criterios de éxito técnico deben definirse antes de comenzar la implementación — una métrica que no puede medirse durante el desarrollo no puede garantizarse en producción.

Módulo 12 – Capítulo 01 – Sección 06

Cierre: el diseño es la decisión más importante del proyecto

El Capítulo 1 estableció el terreno sobre el que se construye el proyecto final. No es un terreno vacío: está delimitado por restricciones de latencia, costo y seguridad que hacen imposible ciertas decisiones y necesarias otras. Está mapeado por una arquitectura de alto nivel que muestra cómo fluyen los datos desde un documento hasta una respuesta, y cómo se organiza el código en un repositorio que cualquier ingeniero puede navegar. Está orientado por criterios de éxito que convierten "el sistema funciona" en "el sistema alcanza faithfulness ≥ 0.85 , latencia P95 $< 3s$ y tasa de bypass $< 5\%$ con evidencia medible". Todo lo que viene después — los ADRs, la implementación, el despliegue, la evaluación — es la ejecución de las decisiones tomadas aquí.

La lección más importante del capítulo de diseño no es técnica: es de proceso. El error más costoso en sistemas de IA no es elegir el modelo equivocado o el framework subóptimo — estos errores son corregibles. El error más costoso es comenzar a implementar sin haber definido los criterios de éxito, sin haber documentado las alternativas evaluadas y sin haber trazado el flujo de datos del sistema completo. Cuando eso ocurre, el equipo construye durante semanas o meses y luego descubre que el sistema no puede evaluarse porque no se definió el golden dataset, no puede auditarse porque los logs no tienen trace_id, y no puede mejorarse porque no hay métricas de referencia contra las cuales medir el cambio. El diseño previo a la implementación no es burocracia — es la inversión que hace posible iterar con evidencia en lugar de con intuición.

Lo que este capítulo no resuelve — deliberadamente — es el detalle de implementación de cada decisión. Esas decisiones se documentan en los ADRs del Capítulo 2, que registran formalmente el razonamiento detrás de cada elección técnica importante: por qué GPT-4o sobre Claude 3.5 Sonnet, por qué text-embedding-3-small sobre text-embedding-3-large, por qué LangGraph sobre LangChain AgentExecutor, y qué controles de seguridad no son negociables. Los ADRs son el puente entre el diseño de alto nivel de este capítulo y la implementación de los capítulos 3 al 9.

Lo que este capítulo resolvió

- **Propósito y contexto:** el proyecto final es un caso de uso enterprise real (asistente técnico para ingenieros de software), con aspectos enterprise presentes (RBAC, auditoría, restricciones de costo) y simplificaciones pedagógicas explícitas (sin integración con legacy systems, sin GDPR completo).
- **Alcance con exclusiones explícitas:** fuentes incluidas (Markdown, PDF técnico, OpenAPI, Confluence), fuentes excluidas (código fuente, emails, Slack), capacidades del agente (solo lectura), y por qué estas exclusiones son decisiones de seguridad tanto como de alcance.
- **Arquitectura con flujos de datos:** el flujo de ingesta asíncrona y el flujo de consulta síncrona como los dos caminos principales que el sistema ejecuta, con cada componente en su capa y la estructura del repositorio que los refleja.
- **Stack con justificación comparativa:** GPT-4o, text-embedding-3-small, Qdrant, LangGraph y FastAPI elegidos con criterios medibles y alternativas evaluadas y descartadas documentadas.
- **Criterios de éxito verificables:** faithfulness ≥ 0.85 , latencia P95 $< 3s$, costo < 0.015 USD/req, bypass $< 5\%$, con la explicación de por qué el gate de CI/CD usa umbrales ligeramente más permisivos (0.82) por razones estadísticas de muestreo.

***Nota del Arquitecto:** El diseño de este capítulo tomará entre medio día y un día de trabajo de un ingeniero senior. Es la inversión de tiempo con mayor retorno de todo el proyecto. Cada hora de diseño previo ahorra entre 3 y 10 horas de refactoring posterior. Si el equipo siente la urgencia de "empezar a codear", ese es exactamente el momento en el que conviene resistir esa urgencia y dedicar otro ciclo a revisar si las restricciones están claras, si los criterios de éxito son medibles y si la arquitectura tiene interfaces limpias entre componentes.*

El Capítulo 2 documenta las decisiones técnicas más importantes del proyecto en cuatro ADRs: elección del modelo fundacional (ADR-001), estrategia de RAG (ADR-002), diseño del agente (ADR-003) y estrategia de seguridad (ADR-004). Estos documentos son la memoria arquitectónica del sistema — la razón por la que, seis meses después del primer despliegue, el equipo puede explicar por qué el sistema funciona como funciona y bajo qué circunstancias tendría sentido cambiarlo.

Para recordar: El diseño de un sistema de IA comienza por las restricciones y los criterios de éxito — la tecnología viene después.

"Good software systems are not built by accumulating features, but by restricting what they do." — Fred Brooks, The Mythical Man-Month